

Nama : Gangsar Wijayanto
NIM : 245150407111022
Kelas : Pengembang Aplikasi Mobile - C

MODUL 3 Event, State, Imperatif-Deklaratif, dan Implisit Intent

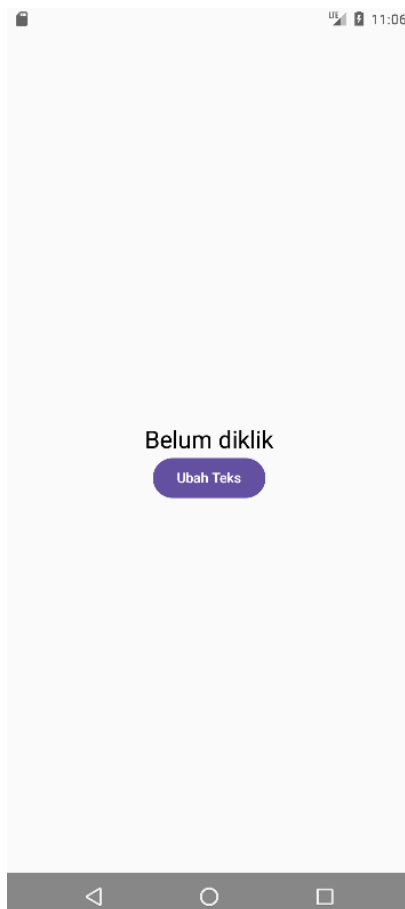
1. Percobaan: Perbandingan Imperatif dan Deklaratif

Jawaban Aktivitas Percobaan:

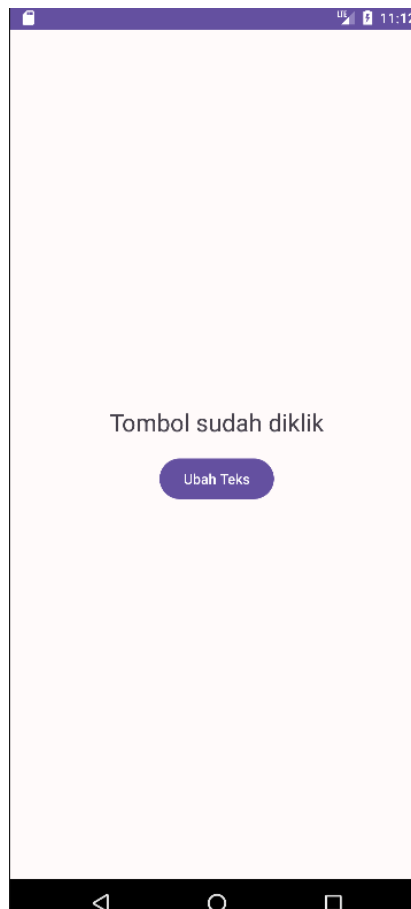
Pertanyaan 1 : Jalankan kedua aplikasi dan ambil screenshot kondisi sebelum serta sesudah tombol diklik.

Project ImperativeXmlApp

Sebelum di click :

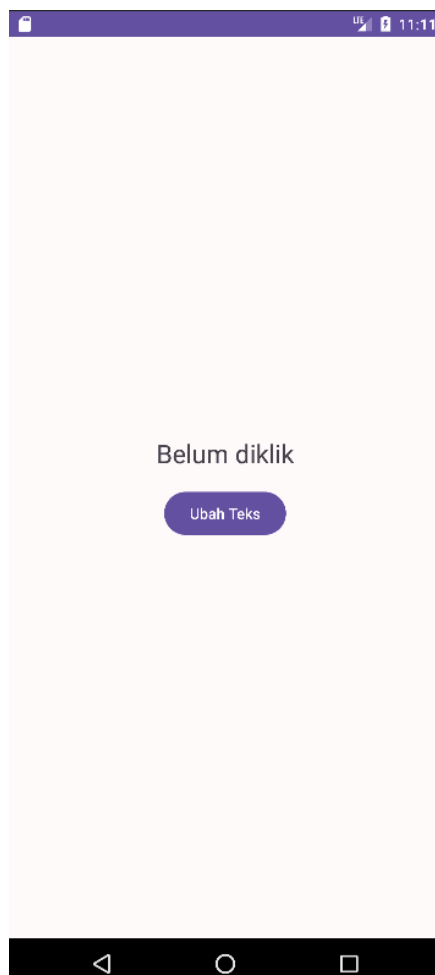


setelah di click :

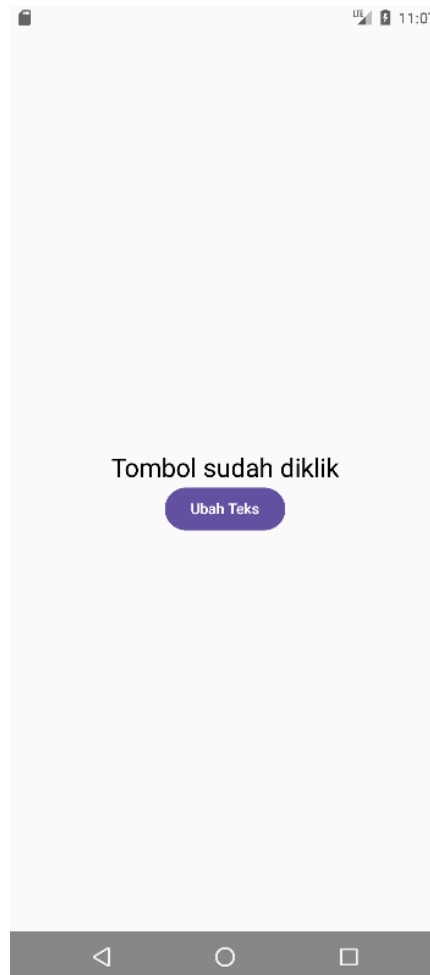


Project DecklarativeXmlApp

Sebelum di click :



setelah di click :



- **Pertanyaan 2:** Pada project XML/View, objek apa yang diubah secara langsung ketika tombol diklik? Tunjukkan baris kodenya.

Objek yang diubah secara langsung pada XML/View adalah TextView. Baris kodenya: `tvMessage.text = "Tombol sudah diklik"`.

- **Pertanyaan 3:** Pada project Compose, nilai apa yang diubah ketika tombol diklik? Tunjukkan baris kodenya.

Pada Compose, nilai yang diubah adalah *state* message. Baris kodenya: `message = "Tombol sudah diklik"`.

- **Pertanyaan 4:** Mengapa pada Compose tidak terdapat perintah seperti `tvMessage.text` atau `setText()`?

Karena Compose menggunakan pendekatan deklaratif. Pengembang tidak mengubah objek Text secara langsung, melainkan mendeklarasikan bahwa teks yang tampil berasal dari suatu *state*.

- **Pertanyaan 5:** Jelaskan alur perubahan pada XML/View dengan pola: event -> View -> perubahan tampilan.

Alur XML/View (Imperatif): *event* -> cari *View* (komponen UI) -> ubah properti *View* secara langsung.

- **Pertanyaan 6:** Jelaskan alur perubahan pada Compose dengan pola: event -> state -> recomposition -> perubahan tampilan.

Alur Compose (Deklaratif): *event* -> ubah *state* -> Compose memperbarui UI melalui *recomposition*.

- **Pertanyaan 7:** Apa yang terjadi jika message diganti menjadi variabel biasa: `var message = "Belum diklik"`? Jelaskan penyebabnya.

Jika message diganti menjadi variabel biasa, nilainya di memori akan berubah, tetapi perubahan tersebut tidak akan otomatis meminta Compose untuk menggambar ulang UI (tidak memicu *recomposition*) sehingga tampilan teks tetap "Belum diklik".

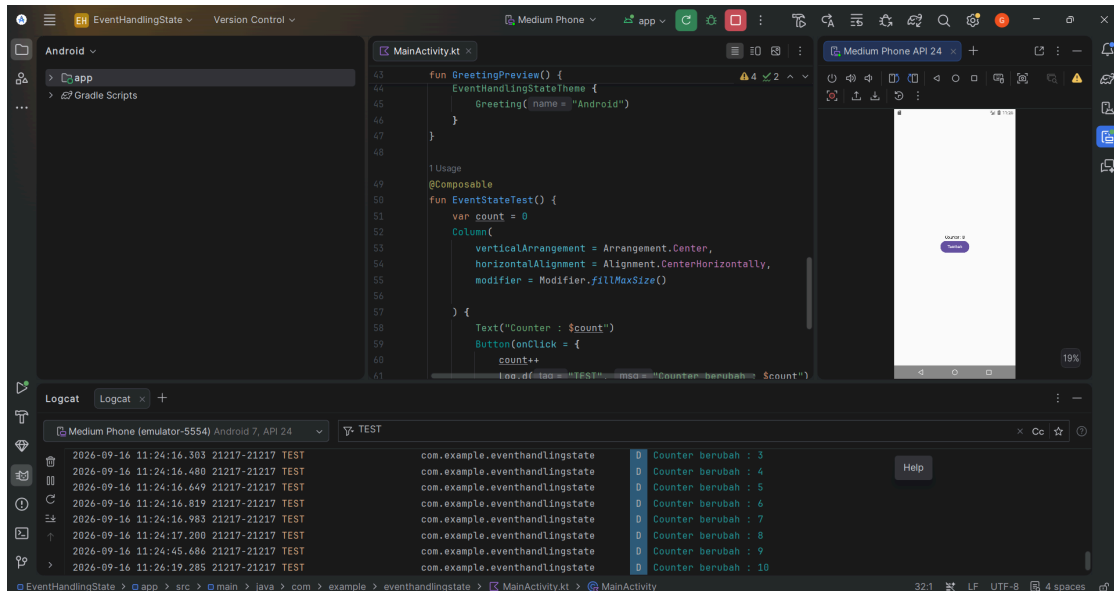
- **Pertanyaan 8 (Kesimpulan):** Tuliskan kesimpulan perbedaan cara berpikir imperatif dan deklaratif menggunakan kalimat Anda sendiri.

Pendekatan imperatif berarti pengembang harus secara manual mencari komponen UI dan memerintahkannya untuk berubah. Sementara itu, pendekatan deklaratif berarti pengembang hanya mengubah data (*state*), lalu UI akan menyesuaikan dan memperbarui dirinya sendiri secara otomatis berdasarkan data terbaru tersebut.

2. Percobaan: Event Handling State

Jawaban Aktivitas Percobaan:

- **Pertanyaan 1 :** Jalankan program, pastikan aplikasi dapat dijalankan dengan baik pada emulator atau perangkat Android fisik. Kemudian buka LogCat, filter LogCat dengan tulisan TEST. Kemudian klik tombol "Tambah" beberapa kali pada aplikasi, ambil satu screenshot tampilan aplikasi setelah tombol "Tambah" diklik beberapa kali.



- **Pertanyaan 2:** Apa yang muncul di LogCat setiap kali tombol "Tambah" diklik?

Di LogCat, setiap tombol diklik akan muncul teks "Counter berubah [angka yang terus bertambah]".

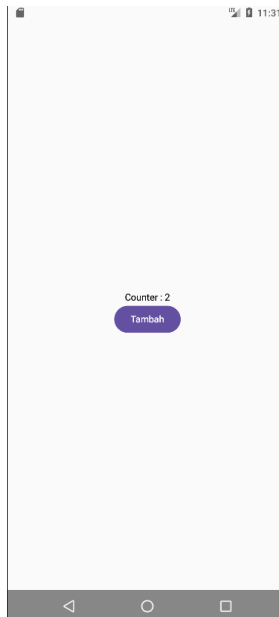
- **Pertanyaan 3:** Apa yang terjadi pada tampilan aplikasi setiap kali tombol "Tambah" diklik?

Pada tampilan aplikasi, tulisan "Counter: 0" **tidak berubah** (tetap menampilkan 0).

- **Pertanyaan 4:** Apa yang menyebabkan hal tersebut terjadi?

Hal tersebut terjadi karena variabel count hanyalah variabel lokal biasa yang tidak bisa diamati perubahannya oleh sistem Jetpack Compose, sehingga UI tidak digambar ulang.

- **Pertanyaan 5:** Perintah Praktikum
- **Pertanyaan 6:** Perintah Praktikum
- **Pertanyaan 7:** Perintah Praktikum
- **Pertanyaan 8:** Perintah Praktikum
- **Pertanyaan 9:** Apa yang muncul di LogCat setiap kali tombol "Tambah" diklik?



(Setelah pakai state) Di LogCat muncul teks "Counter berubah [angka]" sesuai jumlah klik.

- **Pertanyaan 10:** Apa yang terjadi pada tampilan aplikasi setiap kali tombol "Tambah" diklik?

Tampilan aplikasi pada teks "Counter: 0" ikut berubah angkanya sesuai dengan jumlah klik.

- **Pertanyaan 11:** Apa yang menyebabkan hal tersebut terjadi dan berbeda dengan percobaan sebelumnya?

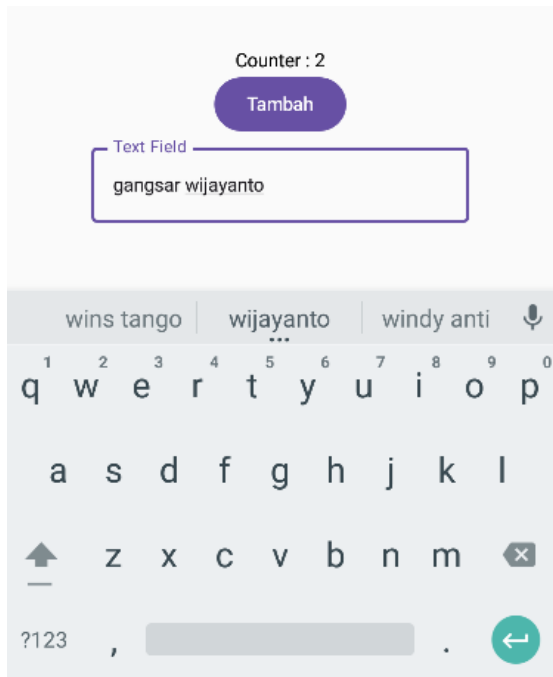
Hal ini berbeda dengan sebelumnya karena count sudah diubah menjadi *state* yang dapat diamati (*mutableStateOf*). Perubahan *state* secara otomatis meminta Compose melakukan penggambaran ulang pada UI.

- **Pertanyaan 12:** Apa yang dimaksud recomposition pada Compose?

Recomposition pada Compose adalah proses di mana Compose memperbarui (menggambar ulang) bagian UI yang bergantung pada *state* saat *state* tersebut nilainya berubah.

- **Pertanyaan 13 :** Perintah Praktikum
- **Pertanyaan 14 :** Perintah Praktikum
- **Pertanyaan 15:** Apa yang muncul di LogCat setiap kali menginputkan huruf di Text Field?

Di LogCat muncul log: "Text berubah: [teks sesuai dengan yang diketikkan]".



```

2026-09-16 11:40:10.338 22110-22110 TEST com.example.eventhandlingstate 0 Counter berubah : 1
2026-09-16 11:40:11.040 22110-22110 TEST com.example.eventhandlingstate 0 Counter berubah : 2
2026-09-16 11:40:12.419 22110-22110 TEST com.example.eventhandlingstate 0 Text berubah : gangsar wijayanto
2026-09-16 11:40:15.058 22110-22110 TEST com.example.eventhandlingstate 0 Text berubah : gangsar wijayanto

```

- **Pertanyaan 16:** Apa yang terjadi pada tampilan aplikasi setiap kali menginputkan huruf di Text Field?

Teks yang diketikkan akan muncul di tampilan *Text Field* di layar.

- **Pertanyaan 17:** Apa yang menyebabkan hal tersebut terjadi?

Hal ini terjadi karena pengetikan memicu *event* `onValueChange` yang memperbarui isi *state* `text`, yang kemudian memicu *recomposition* pada komponen `OutlinedTextField`.

- **Pertanyaan 18:** Sebutkan state apa saja yang ada pada program yang terakhir dibuat?

Terdapat 2 state: `count` (untuk angka) dan `text` (untuk input teks).

- **Pertanyaan 19:** Sebutkan event apa saja yang ada pada program yang terakhir dibuat?

Event yang ada adalah `onClick` (pada tombol) dan `onValueChange` (pada input teks).

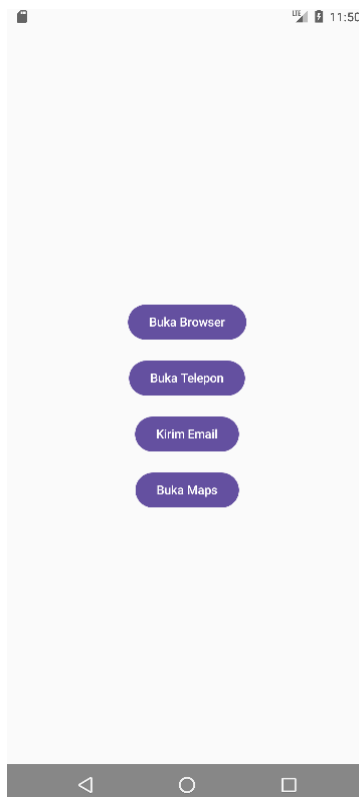
- **Pertanyaan 20:** Kapan event - event tersebut dijalankan?

Event `onClick` dijalankan ketika pengguna menekan tombol "Tambah", sedangkan `onValueChange` dijalankan setiap kali pengguna mengetik atau menghapus huruf di *Text Field*.

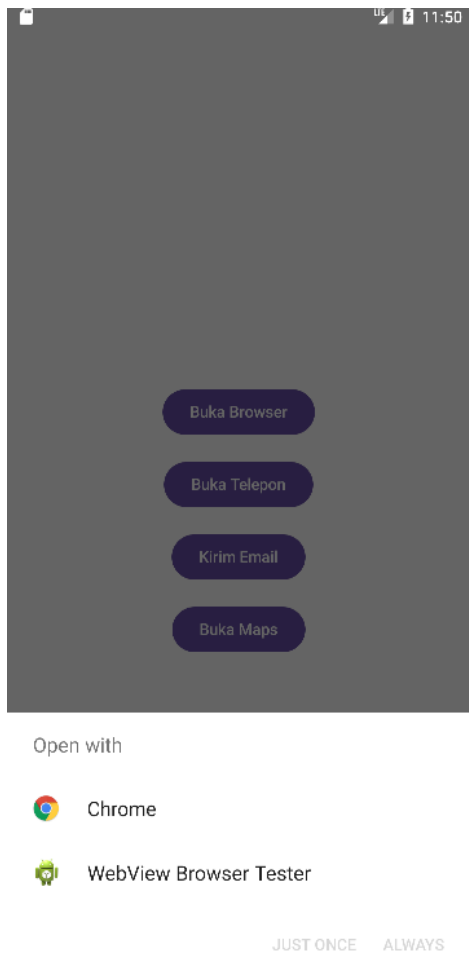
3. Percobaan: Implisit Intent

Jawaban Aktivitas Percobaan:

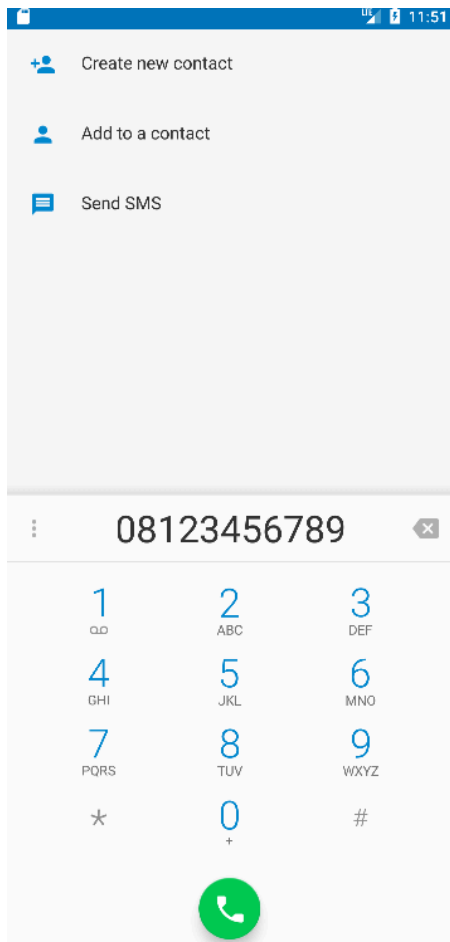
- **Pertanyaan 1** : Jalankan program, pastikan aplikasi dapat dijalankan dengan baik pada emulator atau perangkat Android fisik. Lakukan pengujian pada setiap tombol:



- **Pertanyaan 2** : Klik Buka Browser



- **Pertanyaan 3 :** Klik Buka Telepon

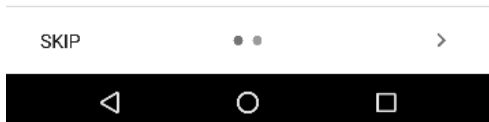


- **Pertanyaan 4 :** Klik Kirim Email



Welcome to Gmail

One app for all of your email



- **Pertanyaan 5 :** Klik Buka Maps



Welcome to Google Maps

By using this application, you agree to our [Terms of Service](#) and [Privacy Policy](#).

ACCEPT & CONTINUE

CLOSE

[Learn more](#) about Google Maps



- **Pertanyaan 6 :** Amati apa yang terjadi pada setiap aksi, kemudian ambil screenshot hasilnya
(Sudah tercantum diatas)
- **Pertanyaan 7 :** Pada kode yang digunakan, terdapat beberapa action seperti:
- **Pertanyaan 8 :** Intent.ACTION_VIEW

```
Button(onClick = {  
    val intent = Intent( action = Intent.ACTION_VIEW).apply {  
        data = Uri.parse( uriString = "geo:-7.250445,112.768845")  
    }  
})
```

- **Pertanyaan 9 :** Intent.ACTION_DIAL

```
Button(onClick = {
    val intent = Intent( action = Intent.ACTION_DIAL).apply {
        data = Uri.parse( uriString = "tel:08123456789")
    }
})
```

- **Pertanyaan 10 :** Intent.ACTION_SENDTO

```
Spacer(modifier = Modifier.height(16.dp))
// Kirim Email (dengan chooser)
Button(onClick = {
    val intent = Intent( action = Intent.ACTION_SENDTO).apply {
        data = Uri.parse( uriString = "mailto:test@gmail.com")
    }
})
```

- **Pertanyaan 11:**
 - Intent.ACTION_VIEW: Digunakan untuk menampilkan sebuah data kepada pengguna. (Misal: membuka browser, membuka peta).
 - Intent.ACTION_DIAL: Digunakan khusus untuk membuka aplikasi panggilan/telepon dan menyisipkan nomor tujuan.
 - Intent.ACTION_SENDTO: Digunakan khusus untuk mengirim pesan atau email dengan menyerahkan data alamat penerima.
- **Pertanyaan 12 :** Pada kode yang digunakan, terdapat beberapa data seperti:
- **Pertanyaan 13 :** data = Uri.parse("mailto:test@gmail.com")
- **pertanyaan 14 :** data = Uri.parse("https://www.google.com")
- **Pertanyaan 15 :** data = Uri.parse("geo:-7.250445,112.768845")
- **Pertanyaan 16:** Apa fungsi data pada intent ?

Fungsi data (seperti *Uri.parse*) pada *Intent* adalah untuk memberikan informasi spesifik (berupa URI/Uniform Resource Identifier) kepada sistem mengenai target apa yang akan dieksekusi. Contoh: nomor HP yang akan dipanggil, alamat email tujuan, atau URL website.

- **Pertanyaan 17:** Apa hubungan Action dan Data pada pilihan aplikasi yang dimunculkan oleh sistem Android

Sistem Android akan menggabungkan informasi antara **Action** (apa yang ingin dilakukan) dengan **Data** (kepada siapa/apa aksi itu ditujukan) untuk memfilter aplikasi mana saja yang cocok dan terpasang di HP pengguna yang mampu menangani *intent* tersebut secara implisit.